

P2372R0

Fixing locale handling in chrono formatters

Published Proposal, 2021-05-01

Authors:

[Victor Zverovich](#)

[Corentin Jabot](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

- 1 The problem
- 2 The solution
- 3 Locale alternative forms
- 4 The "C" locale
- 5 SG16 polls
- 6 Implementation experience
- 7 Impact on existing code
- 8 Wording

References

Informative References

"Mistakes have been made, as all can see and I admit it."

— Ulysses S. Grant

§ 1. The problem

In C++20 "Extending `<chrono>` to Calendars and Time Zones" ([P0355]) and "Text Formatting" ([P0645]) proposals were integrated ([P1361]). Unfortunately during this integration a design issue was missed: `std::format` is locale-

independent by default and provides control over locale via format specifiers but the new formatter specializations for chrono types are localized by default and don't provide such control.

For example:

```
std::locale::global(std::locale("ru_RU"));

std::string s1 = std::format("{}", 4.2);           // s1 == "4.2" (not localized)
std::string s2 = std::format("{:L}", 4.2);         // s2 == "4,2" (localized)

using sec = std::chrono::duration<double>;
std::string s3 = std::format("{:%S}", sec(4.2));    // s3 == "04,200" (localized)
```

In addition to being inconsistent with the design of `std::format`, there is no way to avoid locale other than doing formatting of date and time components manually.

Confusingly, some chrono format specifiers such as `%S` may give an impression that they are locale-independent by having a locale's alternative representation like `%OS` while in fact they are not.

The implementation of [P1361] in [FMT] actually did the right thing and made most chrono specifiers locale-independent by default, for example:

```
using sec = std::chrono::duration<double>;
std::string s = fmt::format("{:%S}", sec(4.2));    // s == "04.200" (not localized)
```

This implementation has been available and actively used in this form for 2+ years. The bug in the specification of chrono formatters in the standard and the mismatch with the actual implementation have only been discovered recently and reported in [LWG3547].

§ 2. The solution

We propose fixing this issue by making chrono formatters locale-independent by default and providing the `L` specifier to opt into localized formatting in the same way as it is done for all other standard formatters (`format.string.std`).

Before	After
<pre>auto s = std::format("{:%S}", sec(4.2)); // s == "04,200"</pre>	<pre>auto s = std::format("{:%S}", sec(4.2)); // s == "04.200"</pre>
<pre>auto s = std::format("{:LS}", sec(4.2)); // throws format_error</pre>	<pre>auto s = std::format("{:LS}", sec(4.2)); // s == "04,200"</pre>

§ 3. Locale alternative forms

Some specifiers (`%d %H %I %m %M %S %u %w %y %z`) produce digits which are not localized (aka they use the Arabic numerals 0123456789) although as we demonstrated earlier `%S` is still using a localized decimal separator. They have an equivalent form (`%0d %0H %0I %0m %0M %0S %0u %0w %0y %0z`) where the numerals can be localized. for example Japanese numerals 〇 一 二 三 四 五 ... can be used as the "alternative representation" by a `ja_JP` locale,

But because the `L` option applies to all specifiers, we do not propose to modify the specifiers.

For example, `"{:L%p%I}"` and `"{:L%p%0I}"` should be valid specifiers producing 午後1 and 午後一 respectively.

Appropriate use of numeral systems for localized numbers and dates requires more work, this paper focuses on a consistent default behavior.

§ 4. The "C" locale

The "C" locale is used in the wording as a way to express locale-independent behavior. The C standard specifies the "C" locale behavior for `strftime` as follows

In the "C" locale, the E and O modifiers are ignored and the replacement strings for the following specifiers are:

```
%a the first three characters of %A.
%A one of 'Sunday', 'Monday', ... , 'Saturday'.
%b the first three characters of %B.
%B one of 'January', 'February', ... , 'December'.
%c equivalent to '%a %b %e %T %Y'.
%p one of 'AM' or 'PM'.
%r equivalent to '%I:%M:%S %p'.
%x equivalent to '%y'.
%X equivalent to %T.
%Z implementation-defined.
```

This makes it possible, as long as the `L` option is not specified, to format dates in environment without locale support (embedded platforms, `constexpr` if someone proposes it, etc).

§ 5. SG16 polls

Poll: LWG3547 raises a valid design defect in [time.format] in C++20.

SF	F	N	A	SA
7	2	2	0	0

Consensus: Strong consensus that this is a design defect.

Poll: The proposed LWG3547 resolution as written should be applied to C++23.

SF	F	N	A	SA
----	---	---	---	----

0	4	2	4	1
---	---	---	---	---

Consensus: No consensus for the resolution

SA motivation: Migrating things embedded in a string literal is very difficult. There are options to deal with this in an additive way. Needless break in backwards with compatibility.

SG16 recognized that this is a design defect but was concerned about this being a breaking change. However, the following facts were not known at the time of the discussion:

- The implementation of [\[P1361\]](#) in [\[FMT\]](#) is locale-independent. This was the only implementation available for 2+ years and was cited as the only source of implementation experience in the paper.
- Both `%S` and `%OS` depend on locale and there is no locale-independent equivalent.
- The chrono formatting in the Microsoft's implementation has only been merged into the main branch on 22 April and has bugs that will require breaking changes.

§ 6. Implementation experience

The `L` specifier has been implemented for durations in the `fmt` library ([\[FMT\]](#)). Additionally, some format specifiers like `S` have never used a locale by default so this was a novel behavior accidentally introduced in C++20:

```
std::locale::global(std::locale("ru_RU"));
using sec = std::chrono::duration<double>;
std::string s = fmt::format("{:%S}", sec(4.2)); // s == "04.200" (not localized)
```

This proposed fix has also been implemented and submitted to the Microsoft standard library.

§ 7. Impact on existing code

Changing the semantics of chrono formatters to be consistent with standard format specifiers ([format.string.std](#)) is a breaking change. At the time of writing the Microsoft's implementation recently merged the chrono formatting into the main branch and is known to be not fully conformant. For example:

```
using sec = std::chrono::duration<double>;
std::string s = std::format("{:%S}", sec(4.2)); // s == "04" (incorrect)
```

§ 8. Wording

All wording is relative to the C++ working draft [\[N4885\]](#).

Update the value of the feature-testing macro `__cpp_lib_format` to the date of adoption in [\[version.syn\]](#):

Change in [\[time.format\]](#):

```
chrono-format-spec:  
fill-and-alignopt widthopt precisionopt Lopt chrono-specsopt
```

2 Each conversion specifier *conversion-spec* is replaced by appropriate characters as described in Table [tab:time.format.spec]; the formats specified in ISO 8601:2004 shall be used where so described. Some of the conversion specifiers depend on ~~the locale that is passed to the formatting function if the latter takes one, or the global locale otherwise.~~ a locale. If the *L* option is used, that locale is the locale that is passed to the formatting function if the latter takes one, or the global locale otherwise. If the *L* option is not used, that locale is the "C" locale. If the formatted object does not contain the information the conversion specifier refers to, an exception of type `format_error` is thrown.

§ References

§ Informative References

[FMT]

Victor Zverovich; et al. [The {fmt} library](https://github.com/fmtlib/fmt). URL: <https://github.com/fmtlib/fmt>

[LWG3547]

Corentin Jabot. [Time formatters should not be locale sensitive by default](https://cplusplus.github.io/LWG/issue3547). URL: <https://cplusplus.github.io/LWG/issue3547>

[N4885]

Thomas Köppe; et al. [Working Draft, Standard for Programming Language C++](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/n4885.pdf). URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/n4885.pdf>

[P0355]

Howard E. Hinnant; Tomasz Kamiński. [Extending to Calendars and Time Zones.](https://wg21.link/p0355). URL: <https://wg21.link/p0355>

[P0645]

Victor Zverovich. [Text Formatting](https://wg21.link/p0645). URL: <https://wg21.link/p0645>

[P1361]

Victor Zverovich; Daniela Engert; Howard E. Hinnant. [Integration of chrono with text formatting](https://wg21.link/p1361). URL: <https://wg21.link/p1361>